

就立刻执行，最后 `count` 的值不会变（别忘了最后执行的是赋值）；但如果是整个赋值完成之后才加 1，最后 `count` 的值会比原来多 1。如果在理解刚才这段话时感到吃力，最好的方法是避开它。

提示 3-13：滥用“++”、“--”、“+=”等可以修改变量值的运算符很容易带来隐蔽的错误。建议每条语句最多只用一次这种运算符，并且所修改的变量在整条语句中只出现一次。

事实上，就算充分理解了这条规则，在实际编程时也可能临时忘记。好在可以利用编译器减少这种错误。用 `-Wall` 命令编译刚才的两个例子，编译器都会给出警告：对 `count` 的运算可能是没有定义的。

3.3 竞赛题目选讲

例题 3-1 TeX 中的引号 (Tex Quotes, UVa 272)^①

在 TeX 中，左双引号是“````”，右双引号是“`''`”。输入一篇包含双引号的文章，你的任务是把它转换成 TeX 的格式。

样例输入：

```
"To be or not to be," quoth the Bard, "that
is the question".
```

样例输出：

```
`To be or not to be,' quoth the Bard, `that
is the question'.
```

【分析】

本题的关键是，如何判断一个双引号是左双引号还是右双引号。方法很简单：使用一个标志变量即可。可是在在此之前，需要解决另外一个问题：输入字符串。

之前学习了使用“`scanf("%s")`”输入字符串，但却不能在本题中使用它，因为它碰到空格或者 TAB 就会停下来。虽然下次调用时会输入下一个字符串，可是不知道两次输入的字符串中间有多少个空格、TAB 甚至换行符。可以用下述两种方法解决这个问题：

第一种方法是使用“`fgetc(fin)`”，它读取一个打开的文件 `fin`，读取一个字符，然后返回一个 `int` 值。为什么返回的是 `int` 而不是 `char` 呢？因为如果文件结束，`fgetc` 将返回一个特殊标记 EOF，它并不是一个 `char`。如果把 `fgetc(fin)` 的返回值强制转换为 `char`，将无法把特殊的 EOF 和普通字符区分开。如果要从标准输入读取一个字符，可以用 `getchar`，它等价于 `fgetc(stdin)`。

提示 3-14：使用 `fgetc(fin)` 可以从打开的文件 `fin` 中读取一个字符。一般情况下应当在检查它

^① 本章最后会介绍 UVaXXX 的含义。

不是 EOF 后再将其转换成 char 值。从标准输入读取一个字符可以用 `getchar`，它等价于 `fgetc(stdin)`。

`fgetc` 和 `getchar` 将读取“下一个字符”，因此需要知道在各种情况下，“下一个字符”是哪个。如果用“`scanf("%d", &n)`”读取整数 `n`，则要是输入 123 后多加了一个空格，用 `getchar` 读取的将是这个空格；如果在“123”之后紧跟着换行，则读取到的将是回车符“`\n`”。

这里有个潜在的陷阱：不同操作系统的回车换行符是不一致的。Windows 是“`\r`”和“`\n`”两个字符，Linux 是“`\n`”，而 MacOS 是“`\r`”。如果在 Windows 下读取 Windows 文件，`fgetc` 和 `getchar` 会把“`\r`”“吃掉”，只剩下“`\n`”；但如果要在 Linux 下读取同样一个文件，它们会忠实地先读取“`\r`”，然后才是“`\n`”。如果编程时不注意，所写程序可能会在某个操作系统上是完美的，但在另一个操作系统上就错得一塌糊涂。当然，比赛的组织方应该避免在 Linux 下使用 Windows 格式的文件，但正如前面所强调过的：选手也应该把自己的程序写得更鲁棒，即容错性更好。

提示 3-15：在使用 `fgetc` 和 `getchar` 时，应该避免写出和操作系统相关的程序。

第二种方法是使用“`fgets(buf, maxn, fin)`”读取完整的一行，其中 `buf` 的声明为 `char buf[maxn]`。这个函数读取不超过 `maxn-1` 个字符，然后在末尾添上结束符“`\0`”，因此不会出现越界的情况。之所以说可以用这个函数读取完整的一行，是因为一旦读到回车符“`\n`”，读取工作将会停止，而这个“`\n`”也会是 `buf` 字符串中最后一个有效字符（再往后就是字符串结束符“`\0`”了）。只有在一种情况下，`buf` 不会以“`\n`”结尾：读到文件结束符，并且文件的最后一个不是以“`\n`”结尾。尽管比赛的组织方应避免这样的情况（和输出文件一样，保证输入文件的每行均以回车符结尾），但正如刚才所说，选手应该把自己的程序写得更鲁棒。

提示 3-16：“`fgets(buf, maxn, fin)`”将读取完整的一行放在字符数组 `buf` 中。应当保证 `buf` 足够存放下文件的一行内容。除了在文件结束前没有遇到“`\n`”这种特殊情况外，`buf` 总是以“`\n`”结尾。当一个字符都没有读到时，`fgets` 返回 `NULL`。

和 `fgetc` 一样，`fgets` 也有一个“标准输入版”`gets`。遗憾的是，`gets` 和它的“兄弟”`fgets` 差别比较大：其用法是 `gets(s)`，没有指明读取的最大字符数。这里就出现了一个潜在的问题：`gets` 将不停地往 `s` 中存储内容，而不管是否存储得下！难道 `gets` 函数不去管 `s` 的可用空间有多少吗？确实如此。

提示 3-17：C 语言并不禁止程序读写“非法内存”。例如，声明的是 `char s[100]`，完全可以赋值 `s[10000]='a'`（甚至 `-Wall` 也不会警告），但后果自负。

正是因为如此，`gets` 已经被废除了，但为了向后兼容，仍然可以使用它。从长远考虑，读者最好不要使用此函数。事实上，在 C11 标准里，`gets` 函数已被正式删除。

提示 3-18：C 语言中的 `gets(s)` 存在缓冲区溢出漏洞，不推荐使用。在 C11 标准里，该函数已被正式删除。

本题的特点是：可以边读边处理，而不需要把输入字符串完整地存下来，因此 `getchar` 是一个不错的选择。下面的代码里还有一个有趣的运算符“?:”，是 `if` 语句的“表达式版”。表达式“`a?b:c`”的含义是：当 `a` 为真时值为 `b`，否则为 `c`。另一个细节是直接用到了赋值语句“`c=getchar()`”的返回值，把它和 `EOF` 进行比较。这样的写法并不多见，但有时能让代码更简洁。

程序 3-5 TeX 中的引号

```
#include<stdio.h>

int main() {
    int c, q = 1;
    while((c = getchar()) != EOF) {
        if(c == '"') { printf("%s", q ? "\"" : "'"); q = !q; }
        else printf("%c", c);
    }
    return 0;
}
```

例题 3-2 WERTYU (WERTYU, UVa10082)

把手放在键盘上时，稍不注意就会往右错一位。这样，输入 `Q` 会变成输入 `W`，输入 `J` 会变成输入 `K` 等。键盘如图 3-2 所示。

输入一个错位后敲出的字符串（所有字母均大写），输出打字员本来想打出的句子。输入保证合法，即一定是错位之后的字符串。例如输入中不会出现大写字母 `A`。

样例输入：

O S, GOMR YPFSU/

样例输出：

I AM FINE TODAY.

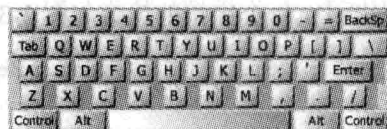


图 3-2 键盘

【分析】

和例题 3-1 一样，每输入一个字符，都可以直接输出一个字符，因此 `getchar` 是输入的理想方法。问题在于：如何进行这样输入输出变换呢？一种方法是使用 `if` 语句或者 `switch` 语句，如“`if(c == 'W') putchar('Q')`”。但很明显，这样做太麻烦。一个较好的方法是使用常量数组，下面是完整程序：

程序 3-6 WERTYU

```
#include<stdio.h>

char s[] = "`1234567890==QWERTYUIOP[]\\ASDFGHJKL;'ZXCVBNM,./";

int main() {
```

```
int i, c;
while((c = getchar()) != EOF) {
    for (i=1; s[i] && s[i]!=c; i++); //找错位之后的字符在常量表中的位置
    if (s[i]) putchar(s[i-1]); //如果找到，则输出它的前一个字符
    else putchar(c);
}
return 0;
}
```

还有其他使用常量数组的方法。例如，构造一个数组 s ，使得对于任意字符 c ， $s[c]$ 的值为 c “左边” 的字符。这个方法也是可行的，但是在程序里输入这样一个 s 数组有些麻烦，还是本题的策略更容易实现。常量数组并不需要指明大小，编译器可以完成计算。

提示 3-19：善用常量数组往往能简化代码。定义常量数组时无须指明大小，编译器会计算。

例题 3-3 回文词 (Palindromes, UVa401)

输入一个字符串，判断它是否为回文串以及镜像串。输入字符串保证不含数字 0。所谓回文串，就是反转以后和原串相同，如 *abba* 和 *madam*。所有镜像串，就是左右镜像之后和原串相同，如 *2S* 和 *3AIAE*。注意，并不是每个字符在镜像之后都能得到一个合法字符。在本题中，每个字符的镜像如图 3-3 所示（空白项表示该字符镜像后不能得到一个合法字符）。

Character	Reverse	Character	Reverse	Character	Reverse
A	A	M	M	Y	Y
B		N		Z	S
C		O	O	1	1
D		P		2	S
E	3	Q		3	E
F		R		4	
G		S	2	5	Z
H	H	T	T	6	
I	I	U	U	7	
J	L	V	V	8	8
K		W	W	9	
L	J	X	X		

图 3-3 镜像字符

输入的每行包含一个字符串（保证只有上述字符。不含空白字符），判断它是否为回文串和镜像串（共 4 种组合）。每组数据之后输出一个空行。

样例输入：

```
NOTAPALINDROME
ISAPALINILAPASI
2A3MEAS
ATOYOTA
```

样例输出:

```
NOTAPALINDROME -- is not a palindrome.
```

```
ISAPALINILAPASI -- is a regular palindrome.
```

```
2A3MEAS -- is a mirrored string.
```

```
ATOYOTA -- is a mirrored palindrome.
```

【分析】

既然不包含空白字符,可以安全地使用 `scanf` 进行输入。回文串和镜像串的判断都不复杂,并且可以一起完成,详见下面的代码。使用常量数组,只用少量代码即可解决这个看上去有些复杂的题目^①。

程序 3-7 回文词

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
const char* rev = "A 3 HIL JM O 2TUVWXY51SE Z 8 ";
const char* msg[] = {"not a palindrome", "a regular palindrome", "a mirrored
string", "a mirrored palindrome"};

char r(char ch) {
    if(isalpha(ch)) return rev[ch - 'A'];
    return rev[ch - '0' + 25];
}

int main() {
    char s[30];
    while(scanf("%s", s) == 1) {
        int len = strlen(s);
        int p = 1, m = 1;
        for(int i = 0; i < (len+1)/2; i++) {
            if(s[i] != s[len-1-i]) p = 0; //不是回文串
            if(r(s[i]) != s[len-1-i]) m = 0; //不是镜像串
        }
        printf("%s -- is %s.\n\n", s, msg[m*2+p]);
    }
    return 0;
}
```

^① 本题是《算法竞赛入门经典》第1版中的一道习题。在第2版写作之时,笔者在网上搜到了很多网友写的本题的题解,不少博主表示本题比较麻烦或者代码冗长,容易写错,故而将此题补充到第2版的例题当中。

本题使用了一个自定义函数 `char r(char ch)`, 参数 `ch` 是一个字符, 返回值是 `ch` 的镜像字符。这是因为该常量数组中前 26 项是各个大写字母的镜像, 而后 10 个是数字 1~9 的镜像 (数字 0 不会出现), 所以需要判断 `ch` 是字母还是数字。函数在第 4 章中会详细讨论, 如果现在理解有困难, 可以等看完第 4 章后回顾此题。

本题用 `isalpha` 来判断字符是否为字母, 类似的还有 `isdigit`、`isprint` 等, 在 `ctype.h` 中定义。由于 ASCII 码表中大写字母、小写字母和数字都是连续的, 如果 `ch` 是大写字母, 则 `ch-'A'` 就是它在字母表中的序号 (A 的序号是 0, B 的序号是 1, 依此类推); 类似地, 如果 `ch` 是数字, 则 `ch-'0'` 就是这个数字的数值本身 (例如 `'5'-'0'=5`)。

另一个有趣的常量数组是 `msg` (事实上, 这是一个字符串数组, 即二维字符数组), 请读者自行理解它的作用。

提示 3-20: 头文件 `ctype.h` 中定义的 `isalpha`、`isdigit`、`isprint` 等工具可以用来判断字符的属性, 而 `toupper`、`tolower` 等工具可以用来转换大小写。如果 `ch` 是大写字母, 则 `ch-'A'` 就是它在字母表中的序号 (A 的序号是 0, B 的序号是 1, 依此类推); 类似地, 如果 `ch` 是数字, 则 `ch-'0'` 就是这个数字的数值本身。

例题 3-4 猜数字游戏的提示 (Master-Mind Hints, UVa 340)

实现一个经典“猜数字”游戏。给定答案序列和用户猜的序列, 统计有多少数字位置正确 (A), 有多少数字在两个序列都出现过但位置不对 (B)。

输入包含多组数据。每组输入第一行为序列长度 n , 第二行是答案序列, 接下来是若干猜测序列。猜测序列全 0 时该组数据结束。 $n=0$ 时输入结束。

样例输入:

```
4
1 3 5 5
1 1 2 3
4 3 3 5
6 5 5 1
6 1 3 5
1 3 5 5
0 0 0 0
10
1 2 2 2 4 5 6 6 6 9
1 2 3 4 5 6 7 8 9 1
1 1 2 2 3 3 4 4 5 5
1 2 1 3 1 5 1 6 1 9
1 2 2 5 5 5 6 6 6 7
0 0 0 0 0 0 0 0 0 0
0
```

样例输出:

```
Game 1:
(1,1)
```

(2,0)

(1,2)

(1,2)

(4,0)

Game 2:

(2,4)

(3,2)

(5,0)

(7,0)

【分析】

直接统计可得 A, 为了求 B, 对于每个数字 (1~9), 统计二者出现的次数 c_1 和 c_2 , 则 $\min(c_1, c_2)$ 就是该数字对 B 的贡献。最后要减去 A 的部分。代码如下:

```
#include<stdio.h>
```

```
#define maxn 1010
```

```
int main() {
```

```
    int n, a[maxn], b[maxn];
```

```
    int kase = 0;
```

```
    while(scanf("%d", &n) == 1 && n) { //n=0 时输入结束
```

```
        printf("Game %d:\n", ++kase);
```

```
        for(int i = 0; i < n; i++) scanf("%d", &a[i]);
```

```
        for(;;) {
```

```
            int A = 0, B = 0;
```

```
            for(int i = 0; i < n; i++) {
```

```
                scanf("%d", &b[i]);
```

```
                if(a[i] == b[i]) A++;
```

```
            }
```

```
            if(b[0] == 0) break; //正常的猜测序列不会有 0, 所以只判断第一个数是否为 0 即可
```

```
            for(int d = 1; d <= 9; d++) {
```

```
                int c1 = 0, c2 = 0; //统计数字 d 在答案序列和猜测序列中各出现多少次
```

```
                for(int i = 0; i < n; i++) {
```

```
                    if(a[i] == d) c1++;
```

```
                    if(b[i] == d) c2++;
```

```
                }
```

```
                if(c1 < c2) B += c1; else B += c2;
```

```
            }
```

```
            printf("    (%d,%d)\n", A, B-A);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

例题 3-5 生成元 (Digit Generator, ACM/ICPC Seoul 2005, UVa1583)

如果 x 加上 x 的各个数字之和得到 y , 就说 x 是 y 的生成元。给出 n ($1 \leq n \leq 100000$), 求最小生成元。无解输出 0。例如, $n=216$, 121, 2005 时的解分别为 198, 0, 1979。

【分析】

本题看起来是个数学题, 实则不然。假设所求生成元为 m 。不难发现 $m < n$ 。换句话说, 只需枚举所有的 $m < n$, 看看有没有哪个数是 n 的生成元。

可惜这样做的效率并不高, 因为每次计算一个 n 的生成元都需要枚举 $n-1$ 个数。有没有更快的方法? 聪明的读者也许已经想到了: 只需一次性枚举 100000 内的所有正整数 m , 标记“ m 加上 m 的各个数字之和得到的数有一个生成元是 m ”, 最后查表即可。

```
#include <stdio.h>
#include <string.h>
#define maxn 100005
int ans[maxn];

int main() {
    int T, n;
    memset(ans, 0, sizeof(ans));
    for(int m = 1; m < maxn; m++) {
        int x = m, y = m;
        while(x > 0) { y += x % 10; x /= 10; }
        if(ans[y] == 0 || m < ans[y]) ans[y] = m;
    }
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        printf("%d\n", ans[n]);
    }
    return 0;
}
```

例题 3-6 环状序列 (Circular Sequence, ACM/ICPC Seoul 2004, UVa1584)

长度为 n 的环状串有 n 种表示法, 分别为从某个位置开始顺时针得到。例如, 图 3-4 的环状串有 10 种表示: CGAGTCAGCT, GAGTCAGCTC, AGTCAGCTCG 等。在这些表示法中, 字典序最小的称为“最小表示”。

输入一个长度为 n ($n \leq 100$) 的环状 DNA 串 (只包含 A、C、G、T 这 4 种字符) 的一种表示法, 你的任务是输出该环状串的最小表示。例如, CTCC 的最小表示是 CCCT, CGAGTCAGCT 的最小表示为 AGCTCGAGTC。

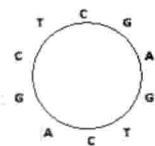


图 3-4 环状串

【分析】

本题出现了一个新概念: 字典序。所谓字典序, 就是字符串在字典中的顺序。一般地,